

ARDMS

Advanced Rescue & Disaster Management System

Project Report and User Manual

A. Abstract

The Advanced Rescue & Disaster Management System (ARDMS) is a complete digital solution designed to save lives during emergencies. It connects people in danger directly with nearby rescue teams and a central command center. By utilizing real-time GPS tracking, instant SOS alerts, and smart task grouping, ARDMS eliminates the confusion and delays typically seen during natural disasters or medical emergencies. This project provides a Web Admin Dashboard for operators, a Public App for victims to ask for help, and a Rescuer App for first responders to navigate exactly where they are needed.

B. Introduction

When a disaster strikes - like a flood, earthquake, or severe medical emergency - every single second matters. Unfortunately, traditional phone calls to emergency numbers can get jammed, and explaining your exact location can be difficult when you are in a panic.

ARDMS solves this problem by using the technology already in our pockets: smartphones. With ARDMS, a person just has to press a single SOS button. Instantly, their exact GPS coordinates, the type of emergency, and their details are sent to a live Web Dashboard. An operator immediately sees this on a map and dispatches a registered Rescuer to that exact spot. The system acts as a digital bridge between those who need help and those who can provide it.

C. Project Description

ARDMS is composed of three main pieces working together simultaneously:

- 1. The Public App: A mobile application for everyday people. It has large, easy-to-read buttons to trigger an SOS for different emergencies (like Medical, Flood, Fire).
- 2. The Rescuer App: A mobile application for emergency responders (paramedics, firefighters, volunteers). It acts like a delivery app, but instead of delivering food, they are delivering help. It gives them a map, routing, and details of the victim.
- 3. The Web Admin Dashboard: A website for the central command team. It shows a live map of all victims and all active rescuers, allowing the operator to play 'matchmaker' and assign the closest rescuer to a victim.

D. Technical Workflow & System Architecture

The system runs on a modern, real-time technology stack. It uses React Native for the mobile apps, React.js for the Web Dashboard, and a Node.js/SQLite backend for the database. Everything communicates instantly using WebSockets, meaning nobody has to refresh their screen to see updates.

1. Web Admin System Explanation

The Web Admin is the 'brain' of the operation. It is a website that the emergency operators look at on their computer screens.

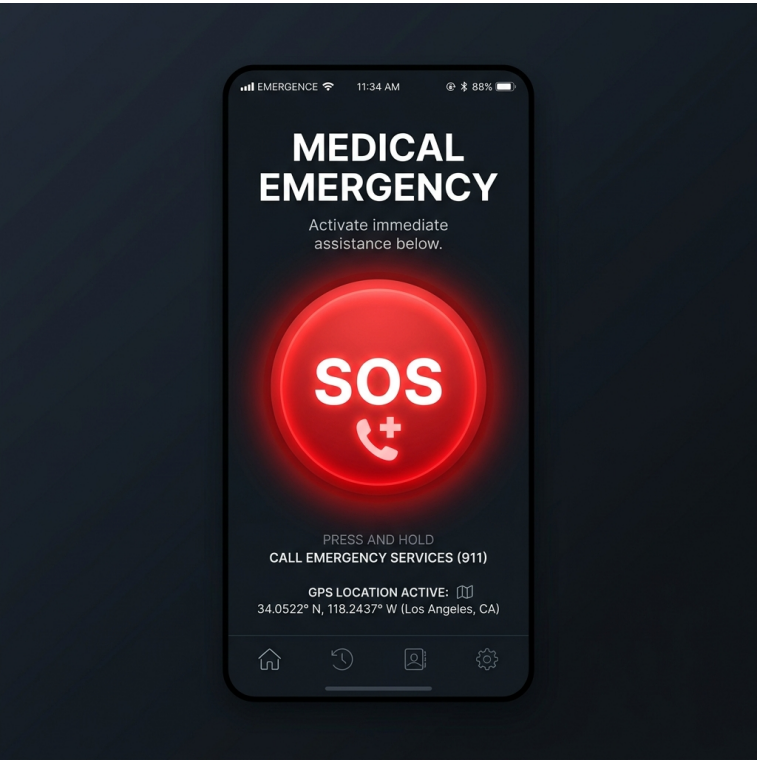
- Live Map View: Operators see red markers for victims and blue markers for rescuers on a live map.
- Task Dispatching: When a new SOS comes in, the operator clicks it and assigns it to an available rescuer.
- Tactical Grouping: If 5 people in the same neighborhood trigger a flood SOS, the admin can group them into one 'Tactical Mission' and send one rescue boat to pick them all up at once.

2. Public Mobile App Explanation

The Public App is designed to be extremely simple, as victims are usually in a state of panic.

- Home Screen: Contains clearly labeled emergency types (Medical, Flood, Fire).
- SOS Trigger: When a category is tapped, a 3-second countdown starts. If not canceled, it grabs the phone's GPS location and sends a 'Critical SOS' to the server.
- Live Tracking: The victim sees a screen saying 'Help is on the way!' and can watch the rescuer's vehicle moving on the map towards them.

[Visual Mockup: Public App Interface]



3. Rescuer Mobile App Explanation

The Rescuer App is built for action and navigation.

- Duty Toggle: The rescuer toggles a switch to say 'I am online and ready to help'.
- Mission Inbox: When the Admin assigns them a task, their phone rings. They see the victim's location and can click 'Accept'.
- Navigation: The app draws a routing line on the map from their current location to the victim.
- Completion: Once they reach the victim and help them, they take a photo as proof, and mark the task as 'Completed'.

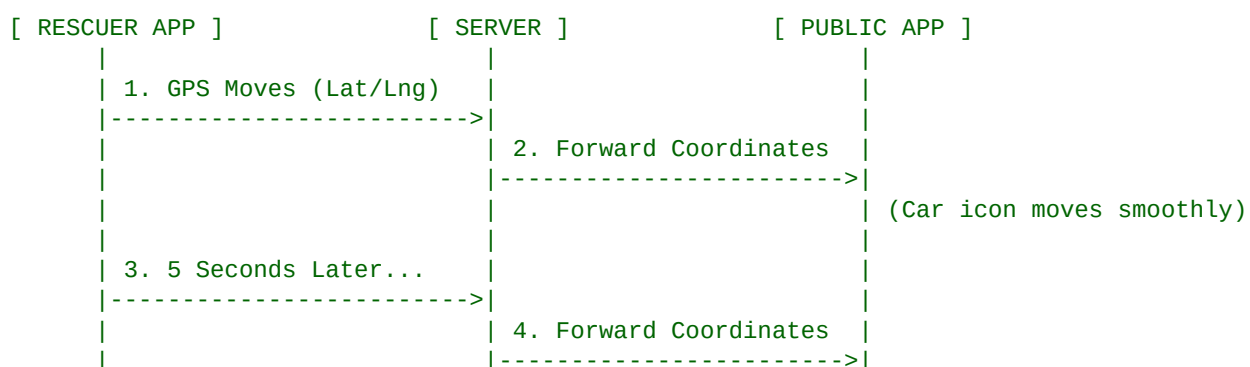
4. Backend & Database Workflow (Signal Flow)

The backend is a Node.js server. It acts as the post office. Below is a visual representation of how a signal flows from the victim to the Admin in a fraction of a second:



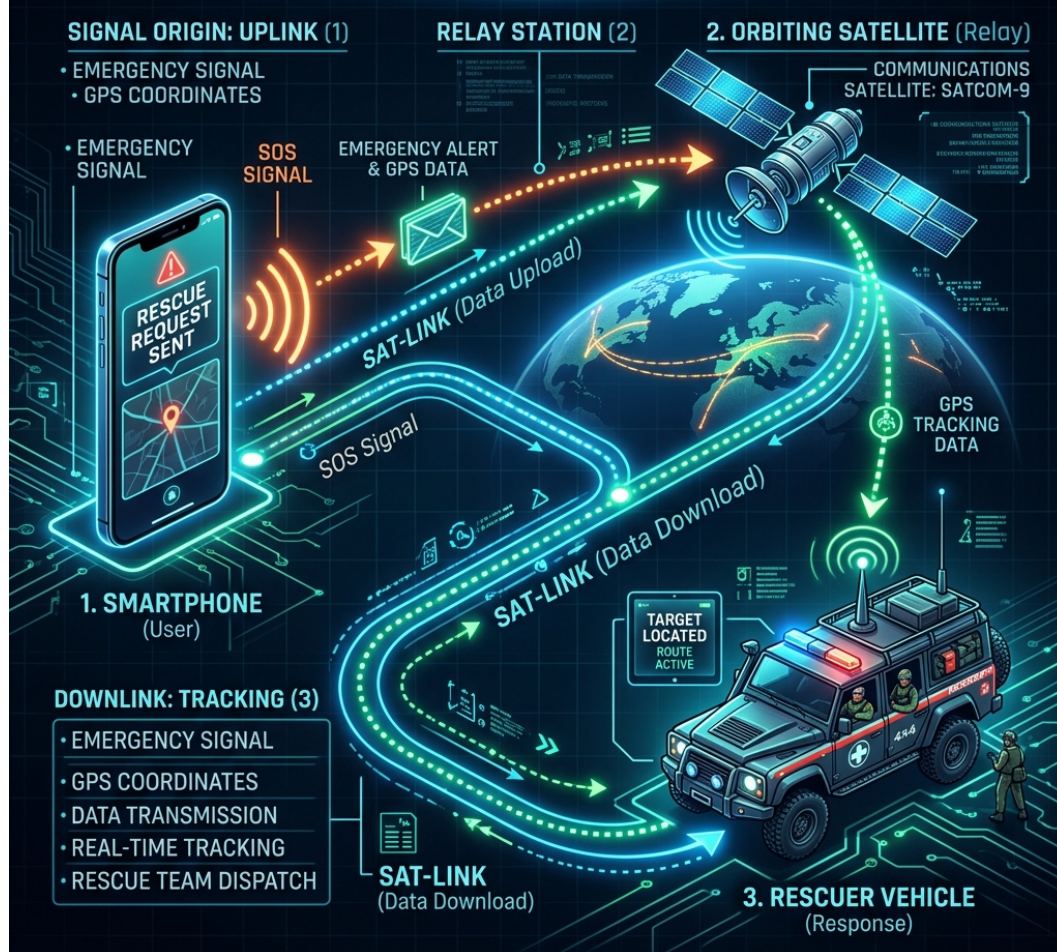
5. Live Tracking & Navigation System (Map Flow)

Live tracking works exactly like Uber. Once a rescuer accepts a mission, their phone's GPS continuously sends their coordinates to the server every few seconds. The server forwards these coordinates to the Victim's Public App.



[Graphical Infographic: Live GPS Tracking Flow]

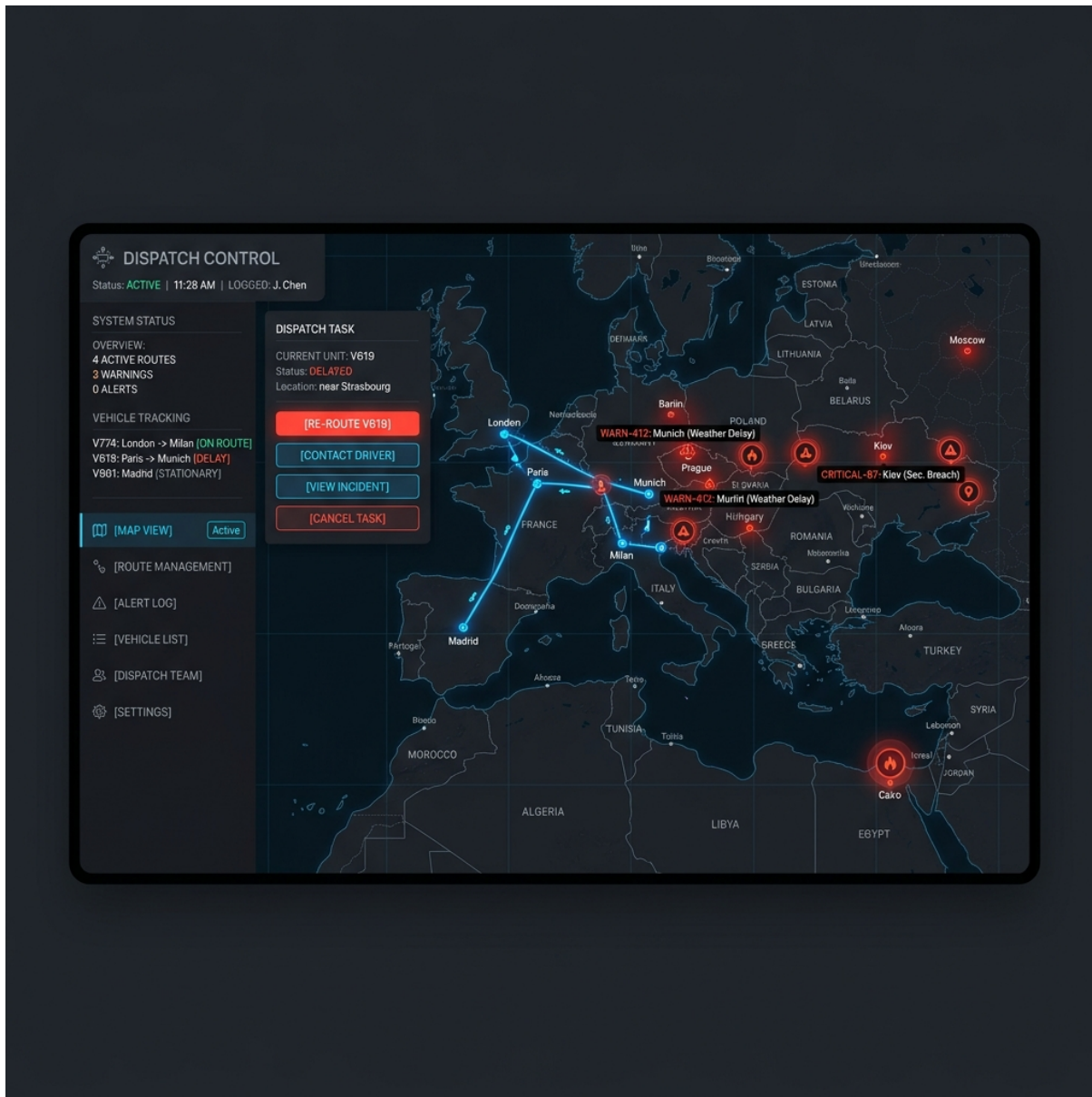
REAL-TIME SATELLITE RESCUE TRACKING



6. Group Task & SOS Management Workflow (Button Operations)

Disasters often happen in clusters. Instead of sending 5 different rescuers to 5 houses on the same flooded street, the Admin uses Group Tasks. Here is the operational flow of the buttons:

[Visual Mockup: Web Admin Control Dashboard]



[WEB ADMIN MAP]

1. [DRAW POLYGON] ---> Selects 3 SOS markers in flooded area
2. [CLICK 'GROUP'] --> Merges into Tactical Mission ID #99
3. [ASSIGN TO JOHN] -> Sends payload to Server

[RESCUER (JOHN)]

4. Phone Rings! -----> Presses [ACCEPT BUTTON]
5. Waypoint 1 -----> Helps Victim -> Presses [MARK COMPLETED]

```
6. Waypoint 2 -----> Helps Victim -> Presses [ MARK COMPLETED ]
7. Waypoint 3 -----> Helps Victim -> Presses [ MARK COMPLETED ]
(Mission Accomplished!)
```

7. Project Structure

The project code is divided into three separate folders:

- public-sos-app/: Contains the React Native code for the victim's app.
- rescuer-app/: Contains the React Native code for the responder's app.
- system-backend/: Contains the Node.js server, the database file (rescue.db), and the Web Admin Dashboard (inside the 'public' folder).

8. Module-wise Functionality

- Authentication Module: Handles secure logins for Admin and Rescuers.
- Location Module: Silently captures GPS coordinates from mobile devices.
- WebSocket Module: Keeps a constant open connection between all apps and the server to push instant updates without refreshing.
- Database Module: Safely stores all history, so even if the server restarts, no SOS requests are lost.

9. Detailed Operational Flow

Here is the step-by-step story of an emergency in ARDMS:

1. Victim opens Public App and taps 'Medical Emergency'.
2. App waits 3 seconds, captures GPS, and POSTs data to Server.
3. Server saves to Database and alerts Web Admin.
4. Admin sees a flashing Red Alert on their map.
5. Admin clicks the alert, selects Rescuer 'John', and clicks 'Assign'.
6. John's Rescuer App rings. John clicks 'Accept'.
7. John follows the blue navigation line on his map to the victim.
8. Victim sees John's vehicle icon moving toward them on their screen.
9. John arrives, helps the victim, clicks 'Complete', and the alert turns green on the Admin dashboard.

10. Button Functions and Screen Explanations

Public App

- Emergency Category Buttons (e.g., Medical, Flood): Initiates the SOS process.
- Cancel Button: Appears during the 3-second countdown to stop accidental triggers.
- Call Support Button: Dials the emergency hotline directly.

Rescuer App

- Go Online Switch: Tells the server the rescuer is available for dispatch.
- Accept/Reject Buttons: Responds to a new mission assignment.
- Mark Completed Button: Closes the mission after the victim is helped.

Web Admin

- Dispatch Button: Sends an SOS task to a selected rescuer.
- Group Button: Merges multiple SOS tasks into a single tactical route.

11. How the Complete System Works

Imagine ordering a pizza. You use an app to say 'I need food here' (Public App). The restaurant sees your order on a computer (Web Admin) and hands it to a delivery driver (Rescuer App). You can then watch the driver's car on a map until they reach your house. ARDMS uses this exact same technology, but instead of delivering a pizza, it delivers life-saving help during emergencies.

12. Localhost Deployment Structure

When running this on your own PC:

- The Node.js Server runs on a local IP address (e.g., 192.168.1.4) on port 3001.
- The Web Admin is accessed by opening <http://localhost:3001> in a web browser.
- The Mobile Apps connect to the server by pointing their API configurations to that same local IP address (<http://192.168.1.4:3001/api>).

13. APK & Web Integration Workflow

The mobile apps are compiled into .apk files for Android devices. Once installed on an Android phone, as long as the phone is on the same WiFi network as the PC (or connected via the internet if deployed globally), the APKs send standard HTTP requests and WebSocket packets to the Web server seamlessly.

14. Real-Time Synchronisation Process

The system uses a library called Socket.IO. When a rescuer moves physically, their phone's GPS changes. The phone sends this new GPS point to the Server. The Server looks at who is assigned to that rescuer, and instantly pushes the new GPS point down to the Victim's phone. This loop happens every 3 to 5 seconds, creating a smooth 'live tracking' experience.

15. Database Structure Overview

The SQLite database uses three main tables to keep track of everything:

- USERS Table: Stores Admin and Rescuer login details, names, and passwords.
- RESCUE_REQUESTS Table: Stores all the SOS alerts. It holds the victim's GPS (latitude/longitude), the type of emergency, the timestamp, and the 'status' (Pending, Assigned, Completed).
- COMMAND_QUEUE Table: Stores the assignments, linking a specific Rescuer ID to a specific Rescue_Request ID.